Bài tập về thừa kế, đa hình

Yêu cầu: Code chạy đúng là chưa đủ. Sinh viên phải giải thích được TẠI SAO nó chạy như vậy khi được hỏi vấn đáp

Bài 1: The constructor chain

Mô tả chi tiết:

1. **Thiết kế Class:**
    - Tạo lớp Person:
        - Thuộc tính: String name, String dob.
        - Constructor mặc định (không tham số): In ra màn hình dòng chữ "1. Person is created".
    - Tạo lớp Employee kế thừa Person:
        - Thuộc tính: double salary.
        - Constructor mặc định: In ra màn hình dòng chữ "2. Employee is created".
    - Tạo lớp Manager kế thừa Employee:
        - Thuộc tính: String department.
        - Constructor mặc định: In ra màn hình dòng chữ "3. Manager is created".
2. **Hàm main:**
    - Trong hàm main, viết đúng **một dòng lệnh**: Manager m = new Manager();.
    - Chạy chương trình và ghi lại kết quả in ra màn hình.
3. **Nâng cao (Bắt buộc):**
    - Sửa lớp Person: Xóa constructor mặc định đi, thay bằng constructor có tham số: public Person(String name) { ... }.
    - **Quan sát:** Lúc này lớp Employee và Manager có báo lỗi không? Tại sao?
    - **Sửa lỗi:** Hãy dùng từ khóa super(...) để sửa lỗi biên dịch cho lớp con.
4. **Ví dụ:**
    - **Input**: (No input)
    - **Output:**
        1. Person is created
        2. Employee is created

3. Manager is created

Bài 2: The polymorphic zoo

Mô tả chi tiết:

1. **Thiết kế Class:**
   - Tạo lớp Animal:
     - Phương thức: public void makeSound() -> In ra "Animal sound".
   - Tạo lớp Dog kế thừa Animal:
     - Ghi đè (Override) hàm makeSound(): In ra "Woof woof".
   - Tạo lớp Cat kế thừa Animal:
     - Ghi đè hàm makeSound(): In ra "Meows meows".
   - Tạo lớp Duck kế thừa Animal:
     - **KHÔNG** ghi đè hàm makeSound().
2. **Thực hành (Trong hàm Main):**
   - Tạo một mảng động vật: Animal[] zoo = new Animal[4];
   - Gán các phần tử ngẫu nhiên:
     - zoo[0] = new Dog();
     - zoo[1] = new Cat();
     - zoo[2] = new Duck();
     - zoo[3] = new Dog();
   - Dùng vòng lặp for (hoặc foreach) duyệt qua mảng zoo. Với mỗi phần tử, gọi lệnh zoo[i].makeSound();.
3. **Yêu cầu quan sát:**
   - Quan sát xem con Vịt (Duck) kêu như thế nào? Tại sao nó lại kêu như vậy?
4. **Ví dụ:**
   - **Input**: (No input)
   - **Output**:

     Woof woof

     Meows meows

     Animal Sound

     Woof woof

Bài 3: The math challenge (Overloading vs Overriding)

Mô tả chi tiết:

1. **Thiết kế Class:**
   - Tạo lớp MathUtils:
     - Hàm public int sum(int a, int b): Trả về a + b.
   - Tạo lớp AdvancedMath kế thừa MathUtils:
     - **Override:** Viết lại hàm sum(int a, int b): Trả về a + b + 10 (Ví dụ logic khác biệt).
     - **Overload:** Viết thêm hàm sum(double a, double b): Trả về a + b.

2. **Thực hành:**
   Trong hàm main:

   ```
   MathUtils m = new AdvancedMath();
   System.out.println(m.sum(5, 5)); // (A)
   // System.out.println(m.sum(5.5, 5.5)); // (B) -> Dòng này có lỗi không?
   ```

   - Quan sát kết quả dòng (A). Nó ra 10 hay 20? Tại sao?
   - Bỏ comment dòng (B). Trình biên dịch có báo lỗi không? Tại sao đối tượng là AdvancedMath (có hàm sum double) mà biến m lại không gọi được?

3. **Ví dụ:**
   - **Input**: (No input)
   - **Output**:

     20

Bài 4: Bẫy ép kiểu

Mục tiêu: Hiểu sự nguy hiểm của Downcasting và lỗi Runtime ClassCastException.

Mô tả chi tiết:

1. **Yêu cầu code (Viết vào hàm Main):** Tại hàm main của Bài 2 - The polymorphic zoo hãy gõ lại đoạn code sau:
```
// Bước 1: Upcasting (An toàn)
Animal a = new Dog(); // Dog kế thừa Animal (lấy từ Bài 2)
// Bước 2: Downcasting (Rủi ro) - Hãy viết dòng này:
Cat c = (Cat) a;
// Bước 3: Gọi hàm
c.makeSound();
```

2. **Thực hành:**
   a. Biên dịch chương trình (javac). Có lỗi không? (Mong đợi: Không lỗi biên dịch).
   b. Chạy chương trình (java). Có lỗi không? Ghi lại tên lỗi (Exception) xuất hiện.
3. **Sửa lỗi:**
   a. Sử dụng cấu trúc if và toán tử instanceof để kiểm tra biến a có đúng là Cat hay không trước khi ép kiểu.
   b. In ra thông báo: "Đây không phải là Mèo!" nếu ép kiểu không hợp lệ.
4. **Ví dụ:**
   a. **Input**: (No input)
   b. **Output**:
   - Trước khi sửa:

     Exception …

   - Sau khi sửa:

     Đây không phải là Mèo!

Bài 5: Hệ thống tính lương Payroll System

Mục tiêu: Thiết kế lớp trừu tượng và áp dụng đa hình để xử lý nghiệp vụ tính toán khác nhau.

Mô tả nghiệp vụ: Một công ty phần mềm cần quản lý trả lương cho nhân viên. Có 2 loại nhân viên Employee:

1. **Nhân viên Full-time** FullTimeEmployee**:** Lương được tính bằng baseSalary + (bonus - penalty).

2. **Nhân viên Part-time** PartTimeEmployee**:** Lương được tính bằng workingHours * hourlyRate.

Yêu cầu thiết kế & Cài đặt:

1. **Phân tích:**
   - Xác định lớp cha Employee. Lớp này chứa các thông tin chung của nhân viên (tên, ngày tháng năm sinh, mã số nhân viên).
   - Xác định các lớp con FullTimeEmployee và PartTimeEmployee. Xác định các thuộc tính và phương thức riêng cần thiết cho từng loại.
   - Vẽ sơ đồ lớp cho hệ thống
2. **Cài đặt:**
   - Viết code Java hiện thực hóa sơ đồ trên.
   - Trong hàm main: Cho một mảng Employee[] chứa thông tin của các nhân viên. In ra bảng lương chi tiết (Tên - Loại NV - Lương thực nhận).
3. **Ví dụ:**
   - **Input**:

     3

     F "Nguyễn Văn A" 1500 200 50

     P "Trần Thị B" 80 10

     F "Lê Văn C" 1400 100 50

     (Dòng đầu tiên nhập vào một số n là số nhân viên. Các dòng tiếp theo mỗi dòng nhập thông tin của từng nhân viên.

     Nếu nhân viên Full-time, thì sẽ có cấu trúc F ["Tên nhân viên"] [baseSalary] [bonus] [penalty]

     Nếu nhân viên Part-time, thì sẽ có cấu trúc P ["Tên nhân viên"] [workingHours] [hourlyRate])

   - **Output**:

     Nguyễn Văn A - Full-time - 1650.0

     Trần Thị B - Part-time - 800.0

Lê Văn C - Full-time - 1450.0

Bài 6: Cửa hàng trực tuyến

Mô tả nghiệp vụ: Xây dựng module tính tiền cho đơn hàng gồm nhiều loại sản phẩm khác nhau:

1. **Sản phẩm điện tử (Electronics):** Giá bán cộng thêm 10% thuế VAT và phí bảo hành (nếu có).
2. **Sản phẩm thực phẩm (Food):** Không chịu thuế. Tuy nhiên, nếu sản phẩm sắp hết hạn (còn < 7 ngày), tự động giảm giá 20%. *(Gợi ý: Dùng java.time.LocalDate để xử lý ngày tháng)*.

Yêu cầu thiết kế & Cài đặt:

1. **Phân tích:**
   - Thiết kế lớp cha Product (mã, tên, giá gốc). Phương thức getFinalPrice() trả về giá gốc.
   - Thiết kế lớp con Electronics và Food áp dụng quan hệ thừa kế. Xác định các thuộc tính và phương thức riêng cần thiết cho từng loại.
   - Vẽ sơ đồ lớp cho hệ thống
2. **Cài đặt:**
   - Tạo đơn hàng (Order) chứa danh sách sản phẩm hỗn hợp.
   - Tính tổng tiền người dùng phải trả.
3. **Ví dụ:**
   - **Input**:

     3

     E "Laptop" 1000 50

     F "Milk" 30 2025-03-15

     F "Bread" 20 2025-03-05

     (Dòng đầu tiên nhập vào một số n là số sản phẩm. Các dòng tiếp theo mỗi dòng nhập thông tin của sản phẩm.

     Nếu là Electronics, thì sẽ có cấu trúc E ["Tên sản phẩm"] [Giá bán] [Phí bảo hành]

Nếu là Food, thì sẽ có cấu trúc F ["Tên sản phẩm"] [Giá bán] [Ngày hết hạn])

- **Output**:

  Laptop - Electronics - 1150.0

  Milk - Food - 30.0

  Bread - Food - 16.0

  Total = 1196.0

Bài 7: Quản lý khách sạn

Bài toán: Một khách sạn có 2 hạng phòng cho thuê:

1. **Phòng Standard:** Giá thuê 500.000đ/đêm.
2. **Phòng VIP:** Giá thuê 2.000.000đ/đêm. (Phòng VIP luôn được miễn phí ăn sáng, không tính thêm tiền).

Tuy nhiên, khách sạn có chính sách mùa vụ:

- Nếu thuê quá 3 đêm, phòng **Standard** được giảm giá 5% trên tổng tiền.
- Phòng **VIP** không giảm giá dù ở bao lâu (do đã có ưu đãi dịch vụ).

Yêu cầu:

- Thiết kế các lớp để quản lý việc tính tiền phòng.
- Viết chương trình nhập vào số đêm ở và loại phòng, sau đó in ra số tiền khách phải trả.

Ví dụ:

- **Input**:
  - Case 1:

    S 3

  - Case 2:

    S 4

- Case 3:

  V 2

(Input sẽ có cấu trúc [Loại phòng] [Số đêm ở]. Trong đó Loại phòng sẽ có hai loại: V - VIP và S - Standard)

- **Output**:
  - Case 1:

    1500000

  - Case 2:

    1900000

  - Case 3:

    4000000

Bài 8: Abstract Class and Interface

Tình huống: Bạn đang lập trình cho một nhà máy chế tạo Robot. Có rất nhiều loại Robot, mỗi loại có cấu tạo cơ bản giống nhau nhưng lại sở hữu những kỹ năng (behavior) rất khác biệt.

1. Khai báo Abstract Class:

Tạo lớp trừu tượng Robot:

- **Thuộc tính:** id (int), modelName (String), batteryLevel (int) (tất cả là private).
- **Constructor:** Nhận vào id và modelName.
- **Instance Method:** void chargeBattery(): tăng pin lên 100%.
- **Final Method:** void showIdentity(): In ra ID và Model - không cho phép lớp con ghi đè để đảm bảo tính bảo mật danh tính.
- **Abstract method:** void performMainTask(): Mỗi robot có một nhiệm vụ chính khác nhau.

2. Khai báo Interfaces:

Vì một Robot có thể sở hữu nhiều kỹ năng cùng lúc, hãy tạo các Interface sau:

- Interface Flyable**:** Phương thức void fly().
- Interface Swimmable**:** Phương thức void swim().
- Interface GPS**:** Phương thức void getCoordinates().

3. Triển khai lớp con kế thừa:

Vì một Robot có thể sở hữu nhiều kỹ năng cùng lúc, hãy tạo các Interface sau:

- Lớp DroneRobot**:** Kế thừa Robot, triển khai (Implement) Flyable và GPS và triển khai các hàm tương ứng.
- Lớp FishRobot**:** Kế thừa Robot, triển khai Swimmable và triển khai các hàm tương ứng.
- Lớp AmphibiousRobot**:** Kế thừa Robot, triển khai **cả 3 Interface**: Flyable, Swimmable, và GPS và triển khai các hàm tương ứng.

4. Hàm Main

- Tạo một danh sách (hoặc mảng) kiểu Robot chứa cả 3 loại Robot trên.
- Duyệt qua danh sách, gọi hàm performMainTask().
- Lấy một đối tượng Robot (nhưng thực chất là DroneRobot) ra khỏi danh sách.
- Thử gọi hàm fly(). Có gọi được không?
- Hãy thực hiện **Downcasting** để robot đó có thể thực hiện kỹ năng bay. Thử tương tự với các loại Robot khác.
- Dùng instanceof để kiểm tra an toàn trước khi ép kiểu: "Nếu robot này có kỹ năng Flyable thì mới cho bay". Kiểm tra tương tự với các loại Robot khác.
- Giả sử bạn có thêm một lớp trừu tượng nữa là ElectronicDevice (Thiết bị điện tử) có phương thức turnOn().
    - Thử cho DroneRobot kế thừa cả Robot và ElectronicDevice. Kết quả là gì? (Gợi ý: Java có cho phép extends 2 lớp không?).
    - Thay vì để ElectronicDevice là lớp trừu tượng, hãy chuyển nó thành **Interface**. Bây giờ hãy cho DroneRobot vừa extends Robot vừa implements Flyable, GPS, ElectronicDevice. Kết quả thế nào?

5. Ví dụ

- **Input**:

  3

  DR 1 Drone-X

  FR 2 Fish-A

  AR 3 Amphibious-Z

  (Dòng đầu tiên nhập vào một số n là số robot.

  Các dòng tiếp theo mỗi dòng nhập thông tin của một robot.

  [Loại Robot] [id] [modelName] Trong đó Loại Robot:

  DR = DroneRobot

  FR = FishRobot

  AR = AmphibiousRobot)

- **Output**:

  Drone-X performing main task

  Drone-X flying

  Drone-X getting coordinates

  Fish-A performing main task

  Fish-A swimming

  Amphibious-Z performing main task

  Amphibious-Z flying

  Amphibious-Z swimming

Amphibious-Z getting coordinates

(Duyệt lần lượt từng phần tử robot trong danh sách,

Gọi performMainTask() cho mọi robot.

Sau đó kiểm tra kỹ năng của robot bằng instanceof:

- Nếu robot instanceof Flyable -> gọi fly()

- Nếu robot instanceof Swimmable -> gọi swim()

- Nếu robot instanceof GPS -> gọi getCoordinates())

Bài 9: Tính tổng chi phí

Mô tả chi tiết:

- **Thiết kế Class:**
  - **Tạo Interface IPayable:**
    - Khai báo phương thức: double getPaymentAmount();
  - **Tạo Lớp Trừu tượng (Abstract) Staff thực thi (implements) IPayable:**
    - Thuộc tính: String id, String name.
    - Phương thức: Viết constructor, getter/setter. *Không* ghi đè getPaymentAmount() (để lớp con tự lo).
  - **Tạo lớp PartTimeStaff kế thừa Staff:**
    - Thuộc tính: int workingHours, double hourlyRate.
    - Bắt buộc ghi đè getPaymentAmount(): Trả về workingHours * hourlyRate.
  - **Tạo lớp Invoice (Hóa đơn) thực thi IPayable:** (Lưu ý: Hóa đơn không phải là Nhân viên, không kế thừa Staff).
    - Thuộc tính: String itemName, int quantity, double pricePerItem.
    - Bắt buộc ghi đè getPaymentAmount(): Trả về quantity * pricePerItem.
- **Hàm main:**
  - Tạo một mảng hoặc danh sách có kiểu IPayable[] payableList.

    - Thêm vào danh sách này cả đối tượng PartTimeStaff và đối tượng Invoice.
    - Tính tổng số tiền mà công ty phải chi trả trong tháng này (Bao gồm cả tiền lương nhân viên và tiền thanh toán hóa đơn).
- **Ví dụ**:
    - **Input**:

      3

      S PT01 NguyenVanA 40 10

      I Laptop 2 500

      S PT02 TranThiB 20 12

      (Dòng đầu tiên nhập vào một số **n** là số đối tượng cần thanh toán.

      Các dòng tiếp theo mỗi dòng nhập thông tin của một đối tượng.

      Nếu là PartTimeStaff:

      S [id] [name] [workingHours] [hourlyRate]

      Nếu là Invoice:

      I [itemName] [quantity] [pricePerItem])

    - **Output**:

      PartTimeStaff NguyenVanA - Payment: 400.0

      Invoice Laptop - Payment: 1000.0

      PartTimeStaff TranThiB - Payment: 240.0

      Total Payment = 1640.0

Bài 10: Quản lý nhân sự và tính thưởng

Mô tả chi tiết:

- **Thiết kế Class:**
  - **Tạo lớp Employee (Nhân viên):**
    - Thuộc tính: String name, double baseSalary.
    - Phương thức: public double calculateBonus() (Mặc định trả về 10% của baseSalary).
  - **Tạo lớp Developer kế thừa Employee:**
    - Thuộc tính: int overtimeHours (Số giờ làm thêm).
    - Ghi đè calculateBonus(): Thưởng = 10% lương cơ bản + (overtimeHours * 200000).
  - **Tạo lớp Tester kế thừa Employee:**
    - Thuộc tính: int bugsFound (Số lỗi tìm được).
    - Ghi đè calculateBonus(): Thưởng = 10% lương cơ bản + (bugsFound * 50000).
- **Hàm main:**
  - Tạo một ArrayList<Employee> và thêm vào danh sách 2 Developer, 2 Tester và 1 Employee bình thường.
  - Duyệt qua danh sách, in ra Tiền thưởng của từng người.
- **Nâng cao (Bắt buộc):**
  - Công ty có chính sách ưu đãi cuối năm: **Chỉ riêng** Developer được tặng thêm một khoá học AWS, và **chỉ riêng** Tester được tặng công cụ Test mới.
  - Yêu cầu: Viết code trong vòng lặp duyệt danh sách, nếu phát hiện nhân viên là Developer thì in ra màn hình "Tặng khóa học AWS", nếu là Tester thì in ra "Tặng tool Test".
  - (Gợi ý: Sử dụng toán tử instanceof để kiểm tra kiểu đối tượng trước khi ép kiểu).
- **Ví dụ**:
  - **Input**:

    5

    E NguyenVanA 10000000

    D TranThiB 12000000 10

    T LeVanC 9000000 20

    D PhamVanD 15000000 5

T HoangThiE 8000000 10

(Dòng đầu tiên nhập vào một số n là số nhân viên

Các dòng tiếp theo mỗi dòng nhập vào thông tin của một nhân viên.

Nếu là Employee thường:

E [name] [baseSalary]

Nếu là Developer:

D [name] [baseSalary] [overtimeHours]

Nếu là Tester:

T [name] [baseSalary] [bugsFound])

- **Output**:

NguyenVanA - Bonus: 1000000.0

TranThiB - Bonus: 3200000.0

Tặng khóa học AWS

LeVanC - Bonus: 1900000.0

Tặng tool Test

PhamVanD - Bonus: 2500000.0

Tặng khóa học AWS

HoangThiE - Bonus: 1300000.0

Tặng tool Test